

Phaser.js 實戰教學：Sprite Sheet 動畫與 Isometric 2.5D 視角

將 2D 平面 Placeholder 升級為《Hades 2》風格等角斜俯視戰鬥視角

在 Phaser.js 中，要將簡單的圓形 Placeholder 升級為具備《Hades 2》感的動作遊戲，需要完成兩個核心步驟：

1. **Isometric (2.5D 等角透視) 座標轉換與圖層排序**：將 2D 平面世界映射到 45 度傾斜等角座標。
2. **Sprite Sheet 載入、剪裁與 8 方向動畫狀態機**：正確設定精靈圖集與動畫監聽。

一、2.5D Isometric (等角透視) 數學與邏輯轉換

《Hades 2》採用 2.5D 菱形等角網格 (Isometric Grid)。邏輯物理計算保持 2D 平面，但視覺繪製時經過矩陣投影變換：

等角轉換核心公式：

- $isoX = (cartX - cartY) * \cos(30^\circ)$
- $isoY = (cartX + cartY) * \sin(30^\circ)$ (通常 X:Y 軸比例為 2:1，投影角度約為 26.565° 或 30°)

1. 座標轉換 Helper 函式

```
// 笛卡兒 2D 座標 -> 等角 2.5D 螢幕座標
function cartToIso(cartX, cartY) {
  const isoX = (cartX - cartY);
  const isoY = (cartX + cartY) / 2;
  return { x: isoX, y: isoY };
}

// 螢幕座標 -> 2D 邏輯座標 (用於滑鼠點擊判定)
function isoToCart(isoX, isoY) {
  const cartX = (2 * isoY + isoX) / 2;
  const cartY = (2 * isoY - isoX) / 2;
  return { x: cartX, y: cartY };
}
```

2. 深度動態排序 (Depth Sorting / Z-Indexing)

在 2.5D 視角中，Y 軸座標越靠下（越靠近螢幕前）的角色/物件必須覆蓋在上方。Phaser 3 提供了非常簡單的寫法：

```
update() {
  // 每一幀根據 Y 座標更新所有可移動物件的 depth
  this.playerContainer.setDepth(this.playerContainer.y);

  this.enemies.forEach(enemy => {
```

```
    if (enemy.active) {
        enemy.setDepth(enemy.y);
    }
});
}
```

二、載入 Sprite Sheet 與建立動畫

1. 在 preload() 中載入 Sprite Sheet

Sprite Sheet 是將角色所有動作（待機、跑步、攻擊）裁切並排列在單一張 PNG 圖片中。載入時必須精確指定每一格 Frame 的寬高：

```
preload() {
    // 載入主角墨利諾厄動畫圖集（假設每格 64x64）
    this.load.spritesheet('melinoe', 'assets/melinoe_spritesheet.png', {
        frameWidth: 64,
        frameHeight: 64
    });

    // 載入等角地板圖塊 Tile
    this.load.image('iso_tile', 'assets/iso_tile_64x32.png');
}
```

2. 在 create() 中定義動畫狀態機 (Anims)

```
create() {
    // 建立「向右跑」動畫
    this.anims.create({
        key: 'run-right',
        frames: this.anims.generateFrameNumbers('melinoe', { start: 0, end: 7 }),
        frameRate: 12,
        repeat: -1 // 無限循環
    });

    // 建立「向下跑」動畫
    this.anims.create({
        key: 'run-down',
        frames: this.anims.generateFrameNumbers('melinoe', { start: 8, end: 15 }),
        frameRate: 12,
        repeat: -1
    });

    // 建立「待機」動畫
    this.anims.create({
        key: 'idle',
        frames: this.anims.generateFrameNumbers('melinoe', { start: 16, end: 19 }),
        frameRate: 6,
        repeat: -1
    });
}
```

```
// 建立「Cast 施法/攻擊」單次動畫
this.anims.create({
  key: 'attack',
  frames: this.anims.generateFrameNumbers('melinoe', { start: 20, end: 25 }),
  frameRate: 18,
  repeat: 0 // 播放一次
});
}
```

三、完整範例程式碼：Phaser.js 替換 Sprite & 8 方向動畫控制器

```
class IsometricPlayerScene extends Phaser.Scene {
  constructor() {
    super('IsometricPlayerScene');
    this.currentAnim = 'idle';
  }

  preload() {
    // 預設圖片路徑 (範例)
    this.load.spritesheet('player_sprite', 'assets/player_sheet.png', {
      frameWidth: 64,
      frameHeight: 64
    });
  }

  create() {
    // 建立 2.5D 物理 Sprite
    this.player = this.physics.add.sprite(400, 300, 'player_sprite');

    // 設定物理碰撞箱 (Isometric 腳底小範圍橢圓/矩形，而非整個 64x64)
    this.player.body.setSize(32, 16);
    this.player.body.setOffset(16, 48); // 將碰撞箱移至角色腳底

    // 初始化動畫
    this.createAnimations();
    this.player.play('idle');

    // 鍵盤輸入監聽
    this.cursors = this.input.keyboard.createCursorKeys();
  }

  createAnimations() {
    this.anims.create({
      key: 'idle',
      frames: this.anims.generateFrameNumbers('player_sprite', { start: 0, end: 3 }),
      frameRate: 6,
      repeat: -1
    });

    this.anims.create({
      key: 'walk-down',
      frames: this.anims.generateFrameNumbers('player_sprite', { start: 4, end: 11 }),

```

```

        frameRate: 12,
        repeat: -1
    });

    this.anims.create({
        key: 'walk-side',
        frames: this.anims.generateFrameNumbers('player_sprite', { start: 12, end: 19 }),
        frameRate: 12,
        repeat: -1
    });

    this.anims.create({
        key: 'walk-up',
        frames: this.anims.generateFrameNumbers('player_sprite', { start: 20, end: 27 }),
        frameRate: 12,
        repeat: -1
    });
}

update() {
    let vx = 0;
    let vy = 0;
    const speed = 160;

    if (this.cursors.left.isDown) vx -= 1;
    if (this.cursors.right.isDown) vx += 1;
    if (this.cursors.up.isDown) vy -= 1;
    if (this.cursors.down.isDown) vy += 1;

    // 正規化方向向量，避免斜向移動速度暴增
    const dir = new Phaser.Math.Vector2(vx, vy).normalize().scale(speed);
    this.player.setVelocity(dir.x, dir.y);

    // ---- 8 方向動畫與 Mirror (FlipX) 邏輯 ----
    if (vx === 0 && vy === 0) {
        this.player.play('idle', true);
    } else {
        if (vy > 0) {
            // 向下走
            this.player.play('walk-down', true);
            this.player.setFlipX(false);
        } else if (vy < 0) {
            // 向上走
            this.player.play('walk-up', true);
            this.player.setFlipX(false);
        } else if (vx !== 0) {
            // 側向走 (左右共用 walk-side, 靠 setFlipX 水平翻轉)
            this.player.play('walk-side', true);
            this.player.setFlipX(vx < 0); // 若往左移動則水平翻轉
        }
    }
}

// 動態深度排序：確保角色在靠近螢幕前方的物件上方
this.player.setDepth(this.player.y);

```

```
}  
}
```

四、重點架構總結表

功能模組	實作重點	Phaser API / 語法
Sprite Sheet 載入	精確定義裁切的單格寬度與高度	<code>this.load.spritesheet(key, url, { frameWidth, frameHeight })</code>
腳底碰撞箱對齊	將預設的全框碰撞箱縮小並移至角色腳底，符合 2.5D 視角	<code>sprite.body.setSize(w, h)</code> <code>sprite.body.setOffset(x, y)</code>
方向動態翻轉	向左與向右的動畫可共用一組，透過 FlipX 節省圖資空間	<code>sprite.setFlipX(true/false)</code>
Z-Index 遮擋排序	將物件層級與 Y 軸座標綁定，實現後方建築遮擋、前方角色覆蓋效果	<code>sprite.setDepth(sprite.y)</code>